Green Software

same work, less energy, cleaner energy

Watch the 4-min video

In one line: measure emissions per unit of work; then cut energy (raise utilisation, remove idle resources, right-size, write efficient software), use cleaner energy (lower-carbon regions, carbon-aware timing for flexible work), and get more years and more work out of each piece of hardware. Most of these also cut cost.

1 A few percent of the world's electricity

Data centres used roughly **1.5%** of the world's electricity in 2024 (about 415 TWh, by the IEA's estimate), and AI is growing that fast: the IEA projects demand could roughly double by 2030. Your company has committed to halving its emissions, and the cloud bill is one of the largest sources. **Green software** means designing systems that do the same work with **less energy**, with **cleaner energy**, and with **less hardware**.

2 Measure first

Cloud providers publish carbon footprint tools (AWS Customer Carbon Footprint Tool, Azure Emissions Impact Dashboard, Google Cloud Carbon Footprint) that estimate emissions per service and region, usually monthly. Open-source tools such as Cloud Carbon Footprint estimate from billing data. For engineering decisions you want a **rate**, not a total, so improvements stay visible as the business grows. The Green Software Foundation's **Software Carbon Intensity** (SCI, published as ISO/IEC 21031:2024) is exactly that:

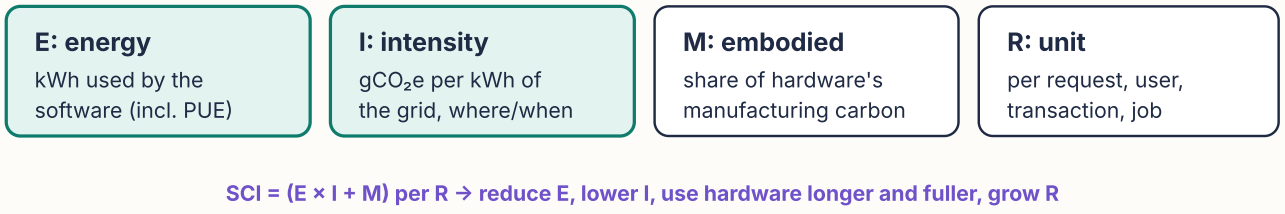


Figure 1. The SCI score. Each term is a lever: energy, the carbon intensity of that energy, embodied carbon of the hardware, and the unit you divide by.

Two accounting details worth knowing. Data-centre overhead is captured by **PUE** (power usage effectiveness: total facility power ÷ IT power); large cloud data centres run around 1.1, while the industry average is about 1.5, which is one reason consolidating into the cloud often helps. And cloud emissions appear in your own reporting as **Scope 3** (purchased services), computed **location-based** (actual grid mix) or **market-based** (after renewable energy purchases); the two can differ a lot, so say which one you mean.

3 Efficiency: the biggest lever

Servers are not energy-proportional: an idle server still draws a large share of its peak power (around half for many older servers, less for modern ones). So **energy per unit of work** falls steeply as utilisation rises.

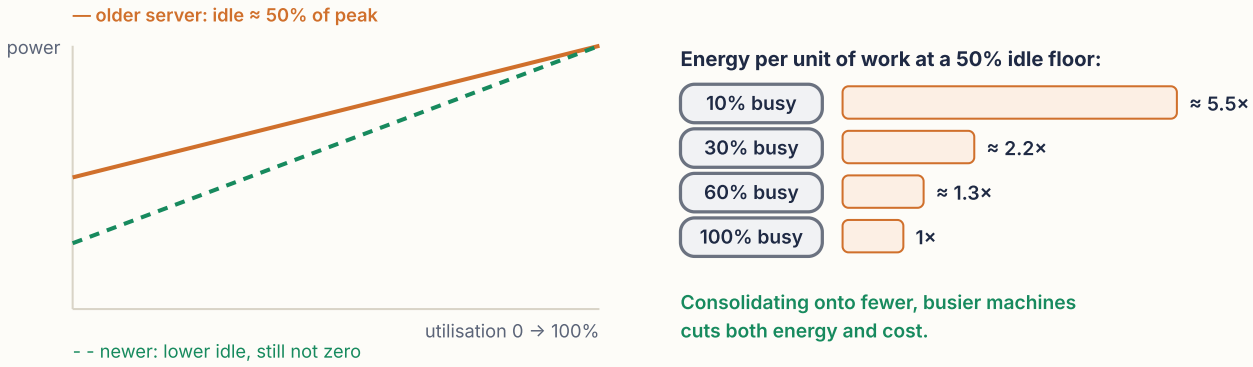


Figure 2. Why utilisation matters. With a 50% idle floor, a machine at 10% load spends about 5.5× the energy per unit of work of a fully used one.

- **Right-size and autoscale** (lessons 58 and 64): fewer, fuller machines.
- **Remove zombies**: idle environments, forgotten clusters, unused replicas. Turning dev/test off outside working hours removes about 64% of their hours.
- **Consolidate** small services onto shared clusters with good bin-packing, and prefer managed or serverless services whose provider runs hardware at high utilisation.
- **Efficient hardware**: newer generations and ARM instances often do more work per watt; for AI, the right accelerator and high batch utilisation matter most.
- **Embodied carbon**: manufacturing hardware is a large part of its lifetime footprint, so running fewer servers for longer, at higher utilisation, beats buying more.

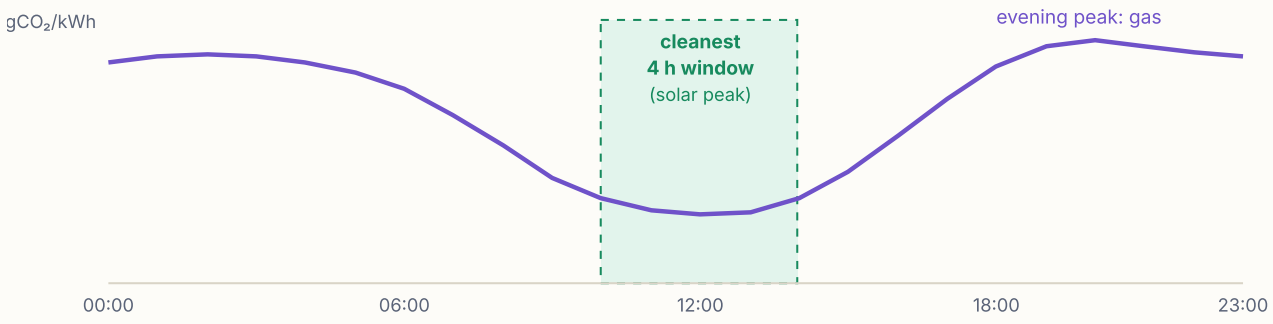
4 Carbon intensity by region

Carbon intensity is grams of CO_2-equivalent per kWh of electricity. It varies hugely by place: grids dominated by hydro, nuclear or wind (Norway, Sweden, Quebec, France) are often under about 50 g/kWh, while coal-heavy grids can exceed 600 g/kWh. The same job can therefore emit **ten times more** depending on the region. When latency, data residency (lesson 46) and cost allow, choose lower-carbon regions for new workloads, especially big batch and training jobs. Providers publish per-region carbon data or "low carbon" indicators to help.

Grid type (illustrative)	Typical intensity	Same 500 kWh monthly job
Mostly hydro / nuclear / wind	~20–60 g/kWh	~10–30 kg CO_2e
Mixed grid with gas and renewables	~200–400 g/kWh	~100–200 kg CO_2e
Coal-heavy	~600–800 g/kWh	~300–400 kg CO_2e

5 Carbon-aware scheduling

Intensity also varies **by hour**: solar lowers it at midday, wind when it's windy, and evening peaks are often met with gas. Flexible work (batch analytics, model training, backups, video transcoding, CI for non-urgent branches) can **shift in time** to the cleanest window before its deadline, or **shift in place** to a region that is cleaner right now. Carbon-intensity APIs (Electricity Maps, WattTime; the Green Software Foundation's Carbon Aware SDK wraps them) provide current values and forecasts that a scheduler can query: "start this 4-hour job in the lowest-intensity window before 06:00 tomorrow". A related idea is **demand shaping**: degrade gracefully when the grid is dirty (lower video quality, defer optional work).



Illustrative day on a grid with lots of solar. Forecasts come from carbon-intensity APIs (e.g. Electricity Maps, WattTime).

Figure 3. Time shifting: the same 4-hour job emits far less in the midday window than starting at the evening peak.

One nuance: **average** intensity says how clean the grid's mix is; **marginal** intensity says which power plant responds when you add load. They can differ, and providers of the data argue for different signals, so pick one consistently and state it.

6 Software choices

- **Do less work**: caching (the greenest request is the one you don't compute), avoiding repeated polling, incremental instead of full recomputation.
- **Efficient code and formats**: better algorithms, compiled or optimised hot paths, compact formats (Parquet over CSV, compression).
- **Smaller AI models** where they are good enough, quantisation, batching, and caching repeated prompts: these change GPU hours by multiples.
- **Store and move less**: retention policies and tiering (lesson 12), fewer copies, less cross-region transfer.
- **Lighter clients**: smaller pages and fewer scripts save energy on billions of user devices, and extend those devices' useful life.

Almost every item also saves money, which makes green work much easier to fund: present it alongside cost (lessons 64–65).

WORKED EXAMPLE: BATCH JOBS IN LOW-CARBON REGIONS

Nightly analytics jobs use about **500 kWh a month** in a coal-heavy region at about 700 g/kWh: roughly **350 kg CO_2e**. The data can legally be processed in another region powered mostly by hydro and wind (residency checked), and the jobs are not latency-sensitive, so they move there. At about 50 g/kWh the same energy is roughly **25 kg**. Model retraining is scheduled with a carbon-intensity forecast to start in the cleanest four-hour window each day, with a deadline so it always finishes before the morning. Emissions for these workloads fall by far more than half; cost barely changes (prices differ by region, and cross-region data transfer was minimised by moving the input data once and keeping it there). The team tracks SCI per job run, so the improvement stays visible as data grows.

COMMON MISTAKES

- Reporting total emissions only, so growth hides efficiency gains.
- Moving workloads to a "green" region without checking residency, latency or transfer cost.
- Ignoring embodied carbon: buying new hardware to save a little energy.
- Treating green as separate from cost and reliability work, instead of the same levers.

7 Key terms

Carbon intensity: gCO_2e emitted per kWh of electricity, by place and time

PUE: facility power ÷ IT power; 1.0 is ideal

Carbon-aware scheduling: shifting flexible work to cleaner times or places

SCI: Software Carbon Intensity: (E × I + M) per unit R

Embodied carbon: emissions from manufacturing and disposing of hardware

Energy proportionality: power use scaling with load; real servers fall short

8 Interview questions

Q1. How can architecture reduce energy use? BASIC

Raise utilisation and remove idle resources, right-size and autoscale, choose lower-carbon regions, shift flexible batch work to cleaner times and places with carbon-aware scheduling, and write efficient software that caches, stores less and avoids unnecessary work.

Q2. How do you track progress? BASIC

Measure emissions and efficiency per unit of work, for example SCI per request or per job, using provider carbon tools and energy estimates.

Q3. What is carbon intensity? BASIC

Grams of CO_2-equivalent per kWh of electricity, which varies by region and by hour with the grid's mix of sources.

Q4. Which workloads would you make carbon-aware first? SCENARIO

Large, flexible, deadline-tolerant ones: batch analytics, training, backups, transcoding; not user-facing latency-critical paths.

Q5. A team wants to move all services to the cleanest region. What do you check? SCENARIO

Data residency and compliance, latency to users, cross-region transfer cost, service availability in that region, and whether batch-only moves get most of the benefit.

Q6. Why can running servers hotter be greener even though it uses more power per server? TRICKY

Idle power is a large floor, so energy per unit of work drops as utilisation rises; fewer, busier servers also need less embodied carbon.

Q7. Why doesn't a renewable-energy purchase make your midday and midnight emissions equal? TRICKY

Market-based accounting nets out purchases annually, but the physical grid's intensity still varies hourly; location-based or hourly signals reveal the difference that carbon-aware scheduling exploits.

Next: Phase 7 · AI and GenAI System Design backendroadmap.com